Applicability of `Lazy` in the JDK and open-source Java software

Architecture review, 14 October 2026. This report evaluates the preview prototype in branch `prototype/lazy-one-class` at commit `615d99c8578f80cba9883d5428f01485b04d84b4`. The contract assessed is `java.lang.util.Lazy`, with `Lazy.md` as its design record. This is a source-level applicability study, not a proposal to migrate the cited projects or a performance benchmark. The prototype is not a committed Java platform API.

Executive assessment

`Lazy` addresses two distinct and recurrent patterns: a cache owns independently initialized state, whereas an updater initializes a caller-owned location. The latter is particularly useful where a receiver already has a nullable field or a fixed array slot and an extra holder would be undesirable. `SERIALIZED` matches double-checked, retry-on-failure initialization; `PUBLISH_ONCE` matches repeatable computations whose losers can be discarded; `PLAIN` matches intentionally uncoordinated memoization. The owned-storage collection forms express fixed index, key, and membership universes, including cached `false` set memberships that a primitive updater cannot store in its reserved `false` state.

The boundary is as important as the positive cases. Many widely used “lazy” implementations are open-ended maps, refreshable registries, asynchronous signal caches, or closeable resources. Replacing these with `Lazy` would change their contract. A high-quality adoption argument should present these exclusions explicitly. Source examples below are pinned to repository commits; the tables count one representative site per external project, not all lazy sites in that project.

Of the **90 selected source sites**, **22 are direct fits**, **38 are conditional**, and **30 have no direct fit**. Among the 60 plausible sites, **50 primarily use an updater**, including **44 field updaters**; the fixed-key map cache and structured-segment updater have no positive example in this sample. These figures measure the audited sites, not the prevalence of lazy patterns across the projects.

Scope and classification

The sample consists of 20 source sites in this JDK checkout, 50 established open-source libraries, and 20 open-source applications. The external-project list was fixed before inspecting each project's site; projects span collections, JSON, networking, persistence, dependency injection, telemetry, test tooling, build tools, servers, and data platforms. “Prominent” denotes a deliberately broad, recognizable project set, not a download or star ranking. The source snapshot is 14 October 2026, with a commit-pinned link on every external row. The JDK rows refer to files in the branch commit above.

For each project, the review selected one implementation site that illustrates a lazy computation or a close alternative. The fastutil row uses its checked-in code-generation template, which defines the generated implementation. The review inspected the read and write path and, where relevant, reset, shutdown, failure, and callback behavior. The labels mean:

Label	Meaning
Direct	No identified change to the site's intended observable contract if its selected mechanism is replaced by the specified `Lazy` construct. A code change and ordinary integration testing would still be required.
Conditional	A plausible use depends on an explicit invariant, acceptance of changed race or exception behavior, or a representation/lifecycle change whose equivalence still needs validation. The condition is stated in the row.
No direct fit	The selected site needs a materially different state machine or capability. Its “closest construct” identifies the missing design dimension; it is not an adoption recommendation.

The assessment is deliberately conservative about `null`, zero, mutable results, reset, asynchronous work, checked exceptions, exception timing, and close-before-initialize races. A direct label is not proof of a mechanical replacement or a speedup. Since the sample chooses a relevant site per project rather than a random source location, its percentages describe **compatibility of these audited sites**, not ecosystem prevalence.

Capability matrix

`Lazy` **construct**	**Storage and granularity**	**Coordination/failure choice**	**Main source pattern**	**Hard boundary**
`ofScalarCache(Supplier)`	One value in a returned supplier	`COMPUTE_ONCE` by default; `PUBLISH_ONCE` explicitly	Static or instance one-time non-null object	Extra holder; no invalidation or close protocol; `COMPUTE_ONCE` retains failures
`ofScalarCache(CacheMode, Function)`	One value in a returned function, **not one per context**	Both cache modes	Noncapturing computation with an access-time receiver or first-caller context	All non-null contexts share one retained winner, which the application must accept; a null context throws even after initialization
`ofListCache`	One value per index in a fixed-size unmodifiable list	Both cache modes, independently per index	Indexed tables, fixed slots	No resizing, mutation, null element, or access-time callback

`Lazy` construct	Storage and granularity	Coordination/failure choice	Main source pattern	Hard boundary
`ofMapCache`	One value per copied key in an unmodifiable fixed-key map	Both cache modes, independently per key	Known finite key universe	No new keys, eviction, invalidation, or null value
`ofSetCache`	One Boolean membership per copied candidate	Both cache modes, independently per candidate	Deferred predicate over a known candidate set	No new candidates or mutable membership; traversal may evaluate candidates
Static/instance field updater	One caller-owned field, optionally per receiver	`PUBLISH_ONCE`, `SERIALIZED`, or `PLAIN` through builder; conveniences fix `PUBLISH_ONCE`	Nullable field sentinel, double-checked singleton, holder-free per-object memo	Default value reserved; no direct reset or unmanaged writes; private fields need suitable lookup/module access; `SERIALIZED` has auxiliary coordination state
Direct or field-backed array updater	One caller-owned physical element; any positive rank up to method/array limits	Three updater modes	Fixed slot table, multi-dimensional result array	No default-valued result; no replacement of array path while in use; `PUBLISH_ONCE` discards losing candidates
Memory-segment updater	One caller-owned value selected by layout path; zero-bit pattern is the	`PUBLISH_ONCE` or `PLAIN`; no `SERIALIZED`	Fixed structured segment location	Segment lifetime is external; atomic access/alignment constraints; no dereference path

`Lazy` construct	Storage and granularity	Coordination/failure choice	Main source pattern	Hard boundary
	uninitialized sentinel			
`buildUseSite` updater	Same selected location as declaration-site updater; callback supplied on each call	Chosen updater mode	Access-specific computation with fixed coordinates	Every call supplies a non-null callback, even after initialization; callbacks sharing one location should be interchangeable; no public use-site cache equivalent

For caches, `COMPUTE_ONCE` invokes a computation at most once per location and makes failure sticky; a non-`Error` failure is reported through `NoSuchElementException`, while an initial `Error` is thrown directly and subsequent access reports `InitializationError`. The original throwable is not retained. Cache `PUBLISH_ONCE` can compute concurrently, installs one winner, and retries after failure. Updater `SERIALIZED` also allows retry after failure, while excluding concurrent computation for the same physical location. Updater `PUBLISH_ONCE` may compute duplicate candidates and returns the installed winner on success. A racing call whose computation fails still fails, even if a different call publishes a value. Updater `PLAIN` can overwrite a competitor and supplies no safe-publication guarantee. Cache values must be non-null. Updaters reserve `null`, `false`, integral zero, positive floating-point zero, or a segment value's zero-bit pattern to mean uninitialized; negative floating-point zero and NaN are valid. A reserved result cannot be stored without a non-default wrapper. Null-restricted reference fields and arrays cannot use this sentinel: the former are rejected when the updater is created, and access to an element of the latter fails even if it already holds a value. These distinctions, rather than the word “lazy,” determine the row assessments.

The updater builder defaults to a public lookup; private fields require authorized lookup/module access, supplied explicitly to the builder or obtained for the caller by a caller-sensitive convenience factory. Rank-two and higher Java-array coordinates, array-valued fields, and primitive carriers beyond the ready-made `int` and `long` forms require custom functional-interface shapes. Cache storage strongly retains its computed values; it retains each computing function until all locations have succeeded or recorded permanent failures. Returned cache suppliers, functions, and collections are deliberately not `Serializable`. Both fixed-key and fixed-candidate factories copy their input universe, which may traverse a lazy input set and compute its memberships. Computation uses the retained key or candidate, which can matter if an equal lookup object has distinct identity or if key state changes.

Versatility measures

The report uses four measures rather than a single unexplained score:

1. **Source-site fit:** direct, conditional, or no direct fit, with denominators of 20 JDK sites, 50 library sites, and 20 application sites.

2. **Construct reach:** the number of direct and conditional sites whose *primary* mapped construct is each cache or updater shape. Each row contributes to at most one primary shape; no-fit rows do not count as coverage.
3. **Policy reach:** which of `COMPUTE_ONCE`, cache/updater `PUBLISH_ONCE`, `SERIALIZED`, and `PLAIN` are needed by the direct/conditional examples.
4. **Uncovered dimensions:** one primary reason per no-direct-fit site, with secondary conditions preserved in its assessment. This makes the blocker table additive without pretending each mismatch has only one cause.

The metrics are architectural breadth indicators. They do not estimate throughput, allocation savings, latency, or the number of sites in a full repository. In particular, an updater removes the need for an extra *per-receiver* cache holder, but the amount saved depends on the application's object graph and the updater's own coordination costs.

Existing branch baseline

This checkout already contains `LazyConstant` and `List.ofLazy`, `Map.ofLazy`, and `Set.ofLazy`. A source search found **32** `LazyConstant.of(...)` **declarations across 17 production source files**, excluding the `LazyConstant` API and implementation files and collection Javadoc examples. Examples include charset providers and aliases, locale resources, SSL cipher-suite lists, and JSON parsed values. This is evidence of already identified one-time scalar uses **in this prototype branch**, not a count from released JDKs. `Lazy.ofScalarCache(COMPUTE_ONCE, ...)` overlaps that role; the new architectural coverage is the explicit cache mode choice and, more markedly, the caller-owned updater shapes and policies. Migration or coexistence of these earlier APIs is outside this survey.

Observed versatility metrics

Each row below counts one audited source site. **Direct** is the conservative high-confidence tier; **Direct + conditional** is the upper envelope of plausible reach if every stated condition is accepted. This purposive sample is not a statistical estimate of Java software at large. Differences among cohorts reflect which sites were selected as well as their code, and a no-fit row does not rule out other uses elsewhere in that project.

Cohort	Sites	Direct	Conditional	No direct fit	Direct + conditional
JDK	20	4 (20%)	11 (55%)	5 (25%)	15 (75%)
Libraries	50	6 (12%)	21 (42%)	23 (46%)	27 (54%)
Applications	20	12 (60%)	6 (30%)	2 (10%)	18 (90%)
All 90 sites	90	22 (24%)	38 (42%)	30 (33%)	60 (67%)

The next table assigns each plausible site one **primary** construct. It does not add a count for alternate possible designs, and it excludes no-fit rows even when they have a named “closest construct.”

Primary construct	Direct	Conditional	JDK	Libraries	Applications	Total plausible sites
scalar cache	2	5	1	3	3	7
list cache	0	2	1	0	1	2

Primary construct	Direct	Conditional	JDK	Libraries	Applications	Total plausible sites
map cache	0	0	0	0	0	0
set cache	1	0	1	0	0	1
field updater	18	26	8	22	14	44
array updater	1	5	4	2	0	6
segment updater	0	0	0	0	0	0
Total	**22**	**38**	**15**	**27**	**18**	**60**

The selected policy is counted separately for cache and updater families, because equal spelling does not make their failure, storage, or return contracts identical. A site with no direct fit does not count as policy evidence.

Policy on plausible site	Direct	Conditional	JDK	Libraries	Applications	Total
Cache `COMPUTE_ONCE`	0	2	0	2	0	2
Cache `PUBLISH_ONCE`	3	5	3	1	4	8
Updater `PUBLISH_ONCE`	1	8	2	6	1	9
Updater `SERIALIZED`	17	13	6	12	12	30
Updater `PLAIN`	1	10	4	6	1	11

Of the 60 plausible sites, **50 (83%)** primarily map to caller-owned updaters rather than owned caches; that is the main incremental reach beyond the branch's existing scalar-oriented `LazyConstant`. **5** plausible rows explicitly need a use-site updater callback, and **1** explicitly select a rank-two Java array location. These are evidence counts, not total occurrences in the projects.

The prototype exposes two storage families; four owned-cache shapes (scalar, fixed list, fixed-key map, fixed-candidate set); field, Java-array, and structured-segment updater locations; two cache policies and three updater policies; declaration-site and use-site updater functions; and nine ready-made use-site accessor interfaces. Zero observed sites for a shape or policy means only that this selection did not yield one.

Primary reasons for no direct fit

Each of the no-direct-fit rows is assigned **one primary blocker** for this table; individual rows can have additional blockers. The counts therefore sum to the no-fit total, unlike a tally of every caveat in every row.

Primary blocker	JDK	Libraries	Applications	Total
Open or changing key universe	2	11	2	15
Reset or replacement of selected state	2	2	0	4

Primary blocker	JDK	Libraries	Applications	Total
Resource creation and close/ownership	1	4	0	5
Asynchronous or reentrant state machine	0	3	0	3
Deferred work without permanent memoization	0	2	0	2
Nullable result plus serialization contract	0	1	0	1
Total	**5**	**23**	**2**	**30**

Audit trail: `J`, `L`, and `A` identify numbered rows in the JDK, library, and application inventories below. Each no-fit row appears in exactly one category:

- **Open or changing key universe:** `J` 17, 18; `L` 2, 5, 7, 11, 12, 20, 21, 24, 28, 31, 37; `A` 4, 18.
- **Reset or replacement of selected state:** `J` 16, 20; `L` 34, 49.
- **Resource creation and close/ownership:** `J` 19; `L` 8, 10, 25, 32.
- **Asynchronous or reentrant state machine:** `L` 15, 16, 23.
- **Deferred work without permanent memoization:** `L` 18, 22.
- **Nullable result plus serialization contract:** `L` 30.

The largest excluded group consists of maps or registries whose keys can appear after construction; a copied fixed universe cannot stand in for their `computeIfAbsent` behavior. Resource closure and reset are separate lifetime concerns. The observed absence of a fitting map-cache example in this sample should be read alongside the site-selection method, not as evidence that fixed-key maps have no use.

Architectural reading of the source evidence

The most persuasive use is a **caller-owned field that uses `null` as its uninitialized sentinel and requires a non-null initialized value**. FontManagerFactory and JdkXmlConfig illustrate serialized singleton construction; Spring Security's service, OpenTelemetry's component name, and gRPC's marshalled bytes show the same shape in libraries. The distinction from `COMPUTE_ONCE` is material: all of these retry after a failed computation, whereas that cache mode records failure permanently. A `SERIALIZED` updater keeps the existing field and models the retry, subject to its separate per-location coordinator and recursion semantics. An existing `synchronized(this)` can also serialize *other* operations or fields; Keycloak's two lazy fields illustrate why monitor identity sometimes needs review.

An updater can also initialize a **mutable container as one value** while leaving later content changes to the container's ordinary API. Jenkins `Actionable`, Hadoop `Configuration`, and Maven's lazy map delegate illustrate this. They are field-updater cases, not uses of `ofMapCache` or `ofListCache`, whose returned collections are unmodifiable and have fixed universes.

The best replacement need not have the same surface shape as the current code. Druid's memoized regex and Trino's memoized output types use Guava suppliers. `COMPUTE_ONCE` would make an exception sticky, unlike Guava's retry. Replacing a private supplier field with a nullable result field and a `SERIALIZED` updater preserves one-at-a-time computation and retry. Druid must read its mutable regex configuration on each attempt until initialization succeeds; both projects must preserve their existing JSON treatment of the cache.

The **publication-only** shape is also real. Protocol Buffers' descriptor retriever loads before installing a descriptor in a volatile field, allowing competing loads. A `PUBLISH_ONCE` field updater expresses the installed winner. Fixed indexed storage is narrower: ImageTypeSpecifier and XSModelImpl are candidates, but their current racy-return or whole-object locking details require explicit validation. The one strong **candidate-membership** example, HandshakeContext, has a finite `NamedGroupSpec` universe and needs to remember both `true` and `false` ; `ofSetCache` represents that without an extra sentinel.

Computation placement is a separate axis from storage shape. JdkXmlConfig uses the first successful initializer's `stax` flag, and Hibernate's collection persister uses the first successful caller's session factory. A use-site updater can accept that input without retaining a capturing lambda in each holder; callers still supply a non-null function on every access, which may itself allocate. JShell's `Startup` instead has a fixed enum key set but takes an access-time `MessageHandler` ; its current map cannot become `ofMapCache` with that callback. A fixed array plus a use-site updater preserves the behavior after a private representation change. This distinction prevents the fixed-key collection factories from being mistaken for general `computeIfAbsent` replacements.

Sentinels and return semantics limit mechanical substitution. The updater's default-valued state excludes primitive zero/false and reference `null` as successful stored results. A non-default wrapper can represent a logical default, but may add allocations and change serialization or API types. Gradle's nullable `AtomicLazy` is a direct scalar-cache fit only with such an internal box or `Optional<T>` . ZipPath is an especially cautionary example: a boxed scalar cache can retain hash zero, yet the source can retain a mutable caller byte array, and the current zero sentinel can trigger recomputation. A permanent scalar cache changes that edge behavior. Even with non-default values, `PLAIN` returns the invoking thread's own candidate; source code that writes and then rereads a field or array slot can return another thread's later candidate. Rows with that pattern are conditional unless the final reread is retained or the race behavior is explicitly judged irrelevant.

Lifetime remains outside `Lazy` **.** FileInputStream, ModulePatcher, and HikariCP have a close-versus-initialize protocol and, in some cases, checked exceptions. `Lazy` has no public close-aware location state. A `PUBLISH_ONCE` computation may also discard a resource produced by a losing thread. Likewise, open-ended, mutable key registries such as Caffeine's manual cache and asynchronous signal caches such as Reactor's `MonoCacheTime` need different state machines. These exclusions should remain visible when presenting the API's scope.

Implications for API review

1. **Keep the cache/updater distinction explicit.** The source examples do not justify treating the modes as interchangeable. The same synchronized lazy field can require retry after failure, while an owned cache's `COMPUTE_ONCE` deliberately makes failure sticky. The two storage families also differ in whether an extra holder is allocated, whether a default value is valid, and who controls location lifetime.
2. **Retain both declaration-site and use-site updater computation.** First-caller parameters and diagnostics appear in actual JDK and library sources. The use-site form needs a clear compatibility rule for functions sharing one physical location. It should not imply one cache entry per function or per context.
3. **Describe** `PLAIN` **as a migration tool only for deliberately racy code.** Its own-result return and lack of safe publication can differ from both a final source-field reread and CAS publication. The branch's acquire/release cache guarantees should not be imputed to this updater mode.
4. **Keep fixed-universe collections narrowly framed.** A lazy map copies its key universe; a lazy set copies its candidate universe. Ordinary `computeIfAbsent` registries, eviction, reset, and mutable collections should not be counted as fits merely because they initialize on demand. Lazy input sets may themselves evaluate memberships

while the universe is copied, and computation uses the retained key or candidate rather than necessarily the equal object used for lookup.

5. **Do not infer performance from fit counts.** The most relevant measurements are allocation per holder, steady-state access latency, contention and losing-computation cost, `SERIALIZED` coordination footprint, and bootstrap footprint. MethodHandleImpl is a particularly sensitive integration test because updater construction itself enters method-handle and classfile machinery; boot, CDS, and AOT tests are necessary before calling that site usable.

Source-site inventory

The JDK paths refer to this checkout at the commit stated above. Each external project row links to its pinned repository revision and to the inspected source line; the assessment is specific to that site.

Twenty JDK source sites

#	Source site	Existing behavior	Closest `Lazy` construct	Assessmen
1	FontManagerFactory.java:44	Volatile double-checked singleton.	static-field updater `SERIALIZED`	**Direct** — construction is single-threaded per field and fail retries.
2	JdkXmlConfig.java:77	First successful initializer's `stax` parameter determines a volatile singleton.	static-field use-site updater `SERIALIZED`	**Direct** — callba supplied per cal preserves first successful initializer's parameter.
3	ImageTypeSpecifier.java:1108	Fixed array of independently lazy image specifiers.	list cache `PUBLISH_ONCE`	**Conditional** — retain `TYPE_CUSTOM` validation and establish that changing the current plain rac whose final slot read can return another thread's object, to a CAS winner is acceptable.
4	MethodHandleImpl.java:90	Per-array-class `MethodHandle[]` computes	direct-array use-site updater `PUBLISH_ONCE`	**Conditional** — callback needs access-time

#	Source site	Existing behavior	Closest `Lazy` construct	Assessmen
		outside lock then synchronizes installation.		`arrayClass` a `access` ; upda generation ente method-handle/ClassFil machinery, so boot, CDS, and AOT compatibili need proof.
5	MethodImpl.java:92	Instance `attributes` field lazily parses classfile data.	instance-field updater `PLAIN`	**Conditional** — preserve the fin field reread if ex race returns matter; custom attribute mappe can have effects so concurrent duplicate parsin needs validatior
6	HandshakeContext.java:324	Local `EnumMap` memoizes both supported and unsupported named-group outcomes.	set cache `PUBLISH_ONCE`	**Direct** — make one candidate s per existing map lifetime and reta miss-only loggir in the predicate; cached `false` remains a valid result.
7	ZipPath.java:630	Racy `int` hash cache uses zero as an uninitialized sentinel.	scalar cache `PUBLISH_ONCE`	**Conditional** — boxed cache ca retain zero, but some `ZipPath` constructors ret input byte array without copying bytes change, permanently memoizing zero changes behavi It also adds per-path holder and boxing cost.

#	Source site	Existing behavior	Closest `Lazy` construct	Assessmen
8	JavadocLog.java:702	Plain instance field defers source-positions lookup until tool environment exists.	instance-field updater `PLAIN`	**Conditional** — keep the callbac deferred until th environment exi and retain the fi field reread if ex race returns matter.
9	Startup.java:352	Fixed enum-key map computes a `Startup` with that call's `MessageHandler` ; its `EnumMap.computeIfAbsent` has no synchronization.	direct-array use-site updater `PLAIN`	**Direct** — replac the private three key enum map with a fixed arra indexed by enur ordinal and pass `MessageHandl` in the use-site callback; the pla updater preserv each racing call own result and non-null fallback
10	Cursor.java:264	Private 14-slot lazy cursor array coexists with protected compatibility array.	direct-array updater `PLAIN`	**Conditional** — preserve the original racy retu of each caller's candidate and tl separate write t the compatibility array.
11	XSModelImpl.java:390	Synchronized two-dimensional `XSNamedMap[][]` lazy slot.	rank-two array updater `SERIALIZED`	**Conditional** — per-slot locking replaces whole-object locking; independence o grammar state requires validati
12	ConnectionCloseFrame.java:200	Lazily decodes an instance `cachedReason` field.	instance-field updater `PUBLISH_ONCE`	**Conditional** — constructors preseed it and `toString()` writes it, so all

#	Source site	Existing behavior	Closest `Lazy` construct	Assessmen
				post-constructio writes need a consistent route
13	ExplodedImage.java:172	Synchronized lazy directory child-name list.	instance-field updater `SERIALIZED`	**Conditional** — computation als clears directorie and registers ch nodes; those effects must remain coordinated.
14	FileInputStream.java:544	Double-checked lazy `FileChannel`.	instance-field updater `SERIALIZED`	**Conditional** — close-before/during-initialization and checked exceptions nee an external lifecycle protoc
15	ModulePatcher.java:282	Double-checked lazy `ModuleReader`.	instance-field updater `SERIALIZED`	**Conditional** — `close()` can itself force opening, and checked `IOException` must be preserv
16	DateFormatSymbols.java:850	Lazy zone-string array is subsequently set, cloned, and serialized.	scalar/field cache	**No direct fit** — mutable setter a serialization protocol.
17	CacheFSInfo.java:104	Dynamic `ConcurrentHashMap` memoizes file attributes.	map cache	**No direct fit** — arbitrary paths a `clearCache()` invalidation.
18	LCMSProfile.java:58	Native tag lookup through mutable map.	map cache	**No direct fit** — arbitrary keys, nullable native results, and `setTag()` invalidation.

#	Source site	Existing behavior	Closest `Lazy` construct	Assessmen
19	JRTIndex.java:147	Shared file-system resources carry owner counts and close/recreate on last release.	static updater	**No direct fit** — resource lifecyc and reset.
20	Config.java:138	Synchronized Kerberos singleton supports explicit `refresh()` .	static updater	**No direct fit** — `refresh()` replaces the singleton.

Fifty open-source libraries

#	Project and revision	Source site	Existing l
1	google/guava (`2b86a82166`)	Suppliers.java:144	Double-checked loc null; thrown comput
2	ben-manes/caffeine (`6bfe815f0f`)	LocalManualCache.java:62	Per-call mapping fu backing cache.comp arbitrary keys.
3	FasterXML/jackson-core (`707b6dfe57`)	SerializedString.java:101	Volatile null-checke encoding is stored v racing callers can c
4	FasterXML/jackson-databind (`f75dc9039f`)	PropertyName.java:235	Plain lazy field uses MapperConfig, inclu compile a name.
5	google/gson (`e2292f9c9f`)	Gson.java:340	Concurrent adapter thread FutureTypeA recursive type cycle
6	apache/commons-lang (`64375a58ee`)	LazyInitializer.java:142	Sentinel plus volatil locking computes o succeeds and failur
7	apache/commons-collections (`c85b117f34`)	LazyMap.java:133	Missing arbitrary ke and mutates backin possible null values

#	Project and revision	Source site	Existing l
8	apache/commons-io (6ee0e0d8c5)	DeferredFileOutputStream.java:476	Threshold transition file/output stream, c later closes it.
9	apache/commons-text (4acfe820c8)	StringLookupFactory.java:276	Nested class holder immutable lookup m properties on first cl
10	apache/httpcomponents-client (4456b34c78)	HttpClientBuilder.java:866	Each build fills a nul manager option with manager.
11	square/okhttp (5a4b597d11)	Cache.kt:193	DiskLruCache requ paired with writes, i HTTP freshness rul
12	square/retrofit (50b452440d)	Retrofit.java:215	Concurrent map use placeholder; parse f for retry.
13	netty/netty (84c167ea85)	AsciiString.java:1150	Plain null-checked S memoization; array(the cached field.
14	eclipse-vertx/vert.x (1458f6ac27)	VertxImpl.java:553	VarHandle CAS inst EventLoop into a nu reread the winner; s slots.
15	reactor/reactor-core (0d1a22ce5b)	MonoCacheTime.java:123	CAS installs an asy coordinator; caches error signals with TT
16	ReactiveX/RxJava (9529995b0e)	SingleCache.java:47	Atomic work count s asynchronously rep success or error to c
17	vavr-io/vavr (0ba17ba1e1)	Lazy.java:172	Lock serializes one supplier run; null is thrown computation
18	eclipse-collections/eclipse-collections (ff113f40cc)	LazyIterableAdapter.java:85	Each iterator travers iterable; transformat per traversal.

#	Project and revision	Source site	Existing I
19	vigna/fastutil (`e70f36a137`)	OpenHashMap.drv:2407	Generated map tem its entry-set view in field.
20	junit-team/junit5 (`7c6c6de5bc`)	ResolutionCache.java:39	ConcurrentHashMa uses a resolver sup null is not retained.
21	mockito/mockito (`613ab03948`)	MockUtil.java:65	ConcurrentHashMa constructs MockMa loaded classes.
22	assertj/assertj (`8e3f3a3544`)	LazyTextDescription.java:39	Each value() calls it result is deferred, n
23	qos-ch/slf4j (`69d28be9fb`)	LoggerFactory.java:497	Synchronized volati returns substitute pr reentrant init and su reset/fallback.
24	qos-ch/logback (`1ae6acb45f`)	LoggerContext.java:160	Per-parent synchron missing loggers and hierarchy node.
25	apache/logging-log4j2 (`c0de1b8628`)	AbstractManager.java:140	Global lock guards r data update, referer later stop/removal.
26	spring-projects/spring-framework (`4dea36c587`)	`spring-core/.../MethodParameter.java:251-260`	Volatile nullable `Pa` populated on first g call; racing threads
27	spring-projects/spring-security (`ab9c14c12f`)	ClaimConversionService.java:47	Volatile, double-che singleton; failure lea

#	Project and revision	Source site	Existing l
28	spring-projects/spring-data-commons (`10feb2ab4b`)	InstanceCreatorMetadataSupport.java:87	Copy-on-write map Boolean creator me properties arrive.
29	hibernate/hibernate-orm (`fb96aa70f0`)	`hibernate-core/.../CollectionType.java:295–313`	Volatile `Collectic` synchronized doubl computed from the `SessionFactoryI`
30	jOOQ/jOOQ (`e450fb50aa`)	`jOOQ/.../Lazy.java:66–96`	serializable DCL su failures and `null` `writeObject` forc
31	mybatis/mybatis-3 (`4210aa1975`)	ResultLoaderMap.java:52	Adds property loade and removes each c
32	brettwooldridge/HikariCP (`a4d93f4f85`)	HikariDataSource.java:92	Lazily starts a conn synchronization; `c` down.
33	flyway/flyway (`b4251cdbec`)	FeatureDetector.java:43	Plain nullable `Bool` one class-presence
34	liquibase/liquibase (`f58ccf20b2`)	SqlGeneratorFactory.java:49	Synchronized lazy s `reset()` replaces
35	micrometer-metrics/micrometer (`16daacfc86`)	FixedBoundaryVictoriaMetricsHistogram.java:141	`AtomicReference` installs a mutable c

#	Project and revision	Source site	Existing l
			each bucket; losing discarded before inc
36	open-telemetry/opentelemetry-java (`4662b0806c`)	ComponentId.java:45	Volatile double-chec name increments a once.
37	resilience4j/resilience4j (`b7c0069802`)	InMemoryRegistryStore.java:49	Future-per-key map threads wait for one removes failures for supports explicit ren
38	grpc/grpc-java (`6f5fccee9f`)	Metadata.java:938	Volatile double-chec `byte[]` serializati value.
39	protocolbuffers/protobuf (`369cb767b2`)	`java/core/.../GeneratedMessage.java:1931–1948`	`CachedDescripto` loads outside its loc one `FieldDescrip` volatile field.
40	apache/kafka (`abc2906503`)	`clients/.../ProduceRequest.java:109–133`	volatile/synchronize partition record size `clearPartitionR` `209` fills it before d
41	elastic/elasticsearch-java (`f920563fb8`)	`java-client/.../LazyDeserializer.java:32–54`	Per-wrapper volatile a delegate to break cycles.
42	google/dagger (`c7e0004b88`)	ApplicationComponentManager.java:35	Volatile double-chec component uses a s retained `Componen`

#	Project and revision	Source site	Existing I
43	google/guice (b8cab7e4e9)	BindingImpl.java:68	Volatile plain assign permits duplicate pr
44	google/auto (30768a2714)	value/.../MemoizeExtension.java:305–315	emits DCL; strate selects a second bo primitive or nullable
45	mapstruct/mapstruct (211b2be620)	processor/.../Type.java:330–349	Plain boxed Boole isNullMarked me scope on each Typ
46	projectlombok/lombok (41af4b3038)	src/core/.../PostCompiler.java:36–66	synchronized static discovery, with an a diagnostics receiver fallback on IOExce
47	raphw/byte-buddy (4e25942007)	byte-buddy-dep/.../TypeDescription.java:9094–9096	@CachedReturnPl makes first-call getDeclaredFiel synthetic transient fi specifies default-val unsynchronized cac
48	classgraph/classgraph (76dfa65400)	classgraph/.../ClassInfo.java:2217–2245	synchronized first-u a private field-overri

#	Project and revision	Source site	Existing I
49	jboss-javassist/javassist (71d5122469)	src/main/.../CtMethod.java:171–187	lazily caches name-signature, but name clears the same fiel
50	snakeyaml/snakeyaml (7b8b171c0d)	src/main/.../PlatformFeatureDetector.java:16–47	Plain nullable boxed caches the Android check per detector i

Twenty open-source applications

#	Project and revision	Source site	
1	apache/tomcat (fe6ce0b16c)	ReplicatedMap.getLog, L284	Transi sync
2	wildfly/wildfly (91f04fe628)	ScheduleTimerMetaDataEntry.getOperator, L59	Volatil constr
3	jenkinsci/jenkins (1e6ac398bc)	Actionable.getActions, L88	Volatil check
4	SonarSource/sonarqube (336d8907a2)	DefaultsRequestBodyAdvice.hasAnnotatedMember, L71	Conc Boole at runt

#	Project and revision	Source site	
5	keycloak/keycloak (867100dda9)	WebAuthnCredentialProviderFactory.createOrGetObjectConverter, L39	Volatil lockin object
6	apache/cassandra (ec2c276be1)	ClientWarn.State.add, L97	Volatil under
7	apache/hadoop (f41bb2ebdc)	Configuration.putIntoUpdatingResource, L4061	Volatil check prepo
8	apache/hbase (0975495494)	AsyncConnectionImpl.toConnection, L414	Volatil double before
9	apache/flink (371ba17ab9)	AbstractMergedChannelStateHandle.ensureCacheUnmergedHandles, L154	Transi check

#	Project and revision	Source site	
10	apache/spark (52d0a661d5)	SpatialReferenceSystemCache.getInstance, L54	Static check
11	elastic/elasticsearch (41d5e26689)	PipelineConfiguration.serializedSizeInBytes, L191	Volatil measu
12	opensearch-project/OpenSearch (b8ebc0aa18)	SearchShardTask metadata supplier, L71	Uses l its del
13	neo4j/neo4j (54a7dcf7c2)	QueryObfuscationState.all, L48	Volatil comp custor
14	apache/zookeeper (48f7cb00e2)	ZKUtil.getPermString, L275	Static Conc render
15	gradle/gradle (6366fe02df)	AtomicLazy.get, L39	Suppli winne

#	Project and revision	Source site	
16	apache/maven (`f4ddd1829d`)	DefaultProjectBuilder.LazyMap.entrySet, L1362	Volatil lockin
17	JetBrains/intellij-community (`facf403338`)	SingleLazyInstanceSyntaxHighlighterFactory.getSyntaxHighlighter, L16	Plain ignorir
18	apache/nifi (`9e6c18893f`)	RecordPathCache constructor/getCompiled, L27	Caffei evicts
19	apache/druid (`7c89096bd1`)	RegexInputFormat memoized pattern, L63	Guava patter retries
20	trinodb/trino (`2896467bf3`)	PartitioningScheme outputTypes, L66	Guava type li